Daily Quant Digest: 2026-10-13

Today's topics

1. Probability & Statistics: Conditional probability, Bayes and the law of total probability (beyond the textbook)
2. Derivatives: Forwards vs futures (cost of carry, FX forward points, margining, convexity)
3. Algorithms & Data Structures: Prefix sums + hashing ("count P&L windows that sum to exactly K")

Difficulty today: Easy / Easy / Medium. This is day one, so we lay foundations that everything else builds on.

---

1. Probability & Statistics: Conditioning Done Properly

1.1 What conditioning actually is

Fix a probability space $(\Omega, \mathcal{F}, \mathbb{P})$. For an event B with $\mathbb{P}(B) > 0$,

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

The right way to think about this: $\mathbb{P}(\cdot \mid B)$ is a **new probability measure**. You throw away every outcome outside B and renormalise what is left so it sums to 1. Everything you know about probability (additivity, inclusion-exclusion, expectations) holds verbatim under $\mathbb{P}(\cdot \mid B)$. Relative likelihoods of outcomes *inside* B are unchanged; that is the whole content of the definition.

Chain rule. Iterating the definition gives

$$\mathbb{P}(A_1 \cap \cdots \cap A_n) = \mathbb{P}(A_1)\,\mathbb{P}(A_2 \mid A_1) \cdots \mathbb{P}(A_n \mid A_1 \cap \cdots \cap A_{n-1}).$$

This is how you compute "probability that the first three cards are all aces" ($\frac{4}{52} \cdot \frac{3}{51} \cdot \frac{2}{50}$) without thinking.

1.2 Law of total probability (LTP)

If $B_1, \ldots, B_n$ partition Ω (disjoint, union is Ω, each with positive probability), then

$$\mathbb{P}(A) = \sum_i \mathbb{P}(A \mid B_i)\,\mathbb{P}(B_i).$$

Proof sketch. $A = \bigsqcup_i (A \cap B_i)$ is a disjoint union, so by additivity $\mathbb{P}(A) = \sum_i \mathbb{P}(A \cap B_i)$, then apply the definition to each term.

Why it matters in interviews: it is the engine of "first-step analysis". Condition on what happens on the first move, write the unknown probability in terms of itself, and solve. (Example: probability a fair coin shows HH before TH. Condition on the first two flips; you will find it is $1/4$, because any T before the first HH means TH must come first.)

1.3 Bayes' theorem, and the form you should actually use

$$\mathbb{P}(H \mid D) = \frac{\mathbb{P}(D \mid H)\,\mathbb{P}(H)}{\sum_j \mathbb{P}(D \mid H_j)\,\mathbb{P}(H_j)}.$$

The denominator is just LTP. In practice, the **odds form** is much faster and less error-prone. For two hypotheses H and H^c:

$$\underbrace{\frac{\mathbb{P}(H \mid D)}{\mathbb{P}(H^c \mid D)}}_{\text{posterior odds}} = \underbrace{\frac{\mathbb{P}(H)}{\mathbb{P}(H^c)}}_{\text{prior odds}} \times \underbrace{\frac{\mathbb{P}(D \mid H)}{\mathbb{P}(D \mid H^c)}}_{\text{likelihood ratio } \Lambda}.$$

The denominator cancels, so you never compute it. Taking logs, **evidence adds**: $\log \text{odds}_{\text{post}} = \log \text{odds}_{\text{prior}} + \sum_k \log \Lambda_k$ when the pieces of evidence are conditionally independent given each hypothesis. This is the mathematical core of naive Bayes classifiers, of sequential testing (Wald's SPRT), and of how a good trader updates on a stream of order-flow signals.

Worked example (base rates). A signal fires for 99% of genuinely informed flow and for 5% of uninformed flow. 1% of flow is informed. Given the signal fired, how likely is the flow informed?

- Prior odds: $1 : 99$.
- Likelihood ratio: $0.99/0.05 = 19.8$.
- Posterior odds: $19.8/99 = 0.2$, i.e. probability $0.2/1.2 = 1/6 \approx 16.7\%$.

Two conditionally independent firings: odds $0.2 \times 19.8 = 3.96$, probability $3.96/4.96 \approx 79.8\%$. The intuition: a 99%-accurate signal is weak evidence when the base rate is 1%, but evidence compounds multiplicatively.

1.4 Independence vs conditional independence

A and B are independent if $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$. They are conditionally independent given C if the same holds under $\mathbb{P}(\cdot \mid C)$. **Neither implies the other.**

- *Conditionally independent but not independent:* pick a coin that is either fair or two-headed, then flip twice. Given the coin, the flips are independent. Unconditionally, the first H makes the two-headed coin more likely, so the flips are positively correlated.
- *Independent but not conditionally independent:* two fair coins, $C = $ "they match". Unconditionally independent; given C, the first determines the second.

The second pattern is **selection bias / collider bias**. Conditioning on a common effect manufactures dependence. In a backtest, "I only looked at stocks that survived" is exactly this.

1.5 Simpson's paradox

Aggregating over a confounder can reverse a comparison. Two strategies, two regimes:

	Calm regime	Volatile regime	Total
Strategy A	81/87 = 93%	192/263 = 73%	273/350 = 78%
Strategy B	234/270 = 87%	55/80 = 69%	289/350 = 83%

A wins in *each* regime but loses overall, because A was deployed mostly in the hard regime. By LTP, the total is a weighted average $\sum_i \mathbb{P}(\text{win} \mid \text{regime}_i, S)\,\mathbb{P}(\text{regime}_i \mid S)$, and the weights differ between strategies. Fix: compare with the *same* weights (stratify), or ask which variable is causally upstream.

1.6 Pitfall: the information-generating process

$\mathbb{P}(A \mid B)$ treats B as "the event that occurred". But in real problems what you observe is "someone *told* me B", and how they chose what to tell you is part of the experiment. Monty Hall is the canonical case: the host's rule (always opens a goat door you did not pick) is what makes switching win with probability $2/3$. If the host opened a random door that *happened* to show a goat, switching wins with probability $1/2$. Same visible facts, different conditioning event. Today's interview question is built around exactly this.

1.7 Pitfall: conditioning on probability-zero events

If X, Y are continuous, $\mathbb{P}(Y \in A \mid X = x)$ cannot come from the ratio formula since $\mathbb{P}(X = x) = 0$. It is defined via densities $f_{Y|X}(y \mid x) = f_{X,Y}(x, y)/f_X(x)$, or abstractly as a conditional expectation given the σ-algebra of X. The **Borel-Kolmogorov paradox** shows the answer depends on which random variable you condition on, not just the set: a uniform point on a sphere, conditioned to lie on a great circle, is uniform on that circle if the circle is the equator (latitude $= 0$), but is *not* uniform if the same circle is described as a meridian (longitude $= 0$ or π). The limit depends on the shape of the shrinking neighbourhoods. Lesson: "given $X = x$" means "given the value of the random variable X", and different random variables carving out the same set give different answers.

1.8 Interview question

> A family has exactly two children. You learn that **at least one of them is a boy born on a Tuesday**. What is the probability that both children are boys?

> Assume each child is independently a boy or girl with probability 1/2, and born on each day of the week with probability 1/7. Follow-up: does the answer change if a parent picked one of their children at random and said “this one is a boy born on Tuesday”?

1.9 Solution

Step 0: set up the sample space. Each child is a pair (sex, day): $2 \times 7 = 14$ equally likely types. Two ordered children (older, younger) give $14^2 = 196$ equally likely outcomes.

Step 1: count the conditioning event B = “at least one child is (Boy, Tue)”. By complement: outcomes with *no* (Boy, Tue) child number $13^2 = 169$, so

$$|B| = 196 - 169 = 27.$$

(Directly: older is B-Tue: 14 outcomes; younger is B-Tue: 14 outcomes; both: 1 counted twice. $14 + 14 - 1 = 27$.)

Step 2: count $A \cap B$, where A = “both boys”. Now each child is one of 7 boy-types, so 49 boy-boy outcomes, of which those with no Tuesday boy number $6^2 = 36$:

$$|A \cap B| = 49 - 36 = 13.$$

(Directly: $7 + 7 - 1 = 13$.)

Step 3: divide. All outcomes are equally likely, so

$$\mathbb{P}(A \mid B) = \frac{13}{27} \approx 0.481.$$

Why it is not 1/3 or 1/2. With “at least one boy” the answer is $1/3$ (BB, BG, GB equally likely). With “the older child is a boy” it is $1/2$. “Boy born on Tuesday” is a *more specific* label: it almost singles out one child, so the answer moves toward $1/2$. The only reason it is not exactly $1/2$ is the double-counted outcome where both children are Tuesday boys.

General formula (good sanity check). If the extra attribute has probability p (here $p = 1/7$), then

$$\mathbb{P}(B) = 1 - (1 - p/2)^2 = p - \tfrac{p^2}{4}, \qquad \mathbb{P}(A \cap B) = \tfrac{1}{4}\left(1 - (1-p)^2\right) = \tfrac{2p - p^2}{4},$$

$$\mathbb{P}(A \mid B) = \frac{2p - p^2}{4p - p^2} = \frac{2 - p}{4 - p}.$$

Check limits: $p = 1$ (no information beyond "a boy") gives $1/3$. $p \to 0$ (the attribute identifies a child uniquely) gives $1/2$. $p = 1/7$ gives $\frac{13/7}{27/7} = 13/27$. Matches.

Interview trick (odds form). Hypotheses BB, BG/GB, GG with prior odds $1 : 2 : 1$. Likelihood of "at least one Tuesday boy" under each: BB: $1 - (6/7)^2 = 13/49$; mixed: $1/7 = 7/49$; GG: 0. Posterior odds $1 \cdot 13 : 2 \cdot 7 : 0 = 13 : 14$, so $\mathbb{P}(BB) = 13/27$. Ten seconds once you see it.

Follow-up: the parent volunteers it. Now the observed event is "a randomly chosen child is reported as (Boy, Tue)". Likelihoods:

- BB: chosen child is a boy for sure, Tuesday w.p. $1/7$: likelihood $1/7$.
- Mixed (BG or GB): need to pick the boy $(1/2)$ and he is Tuesday $(1/7)$: $1/14$.
- GG: 0.

Posterior odds $1 \cdot \frac{1}{7} : 2 \cdot \frac{1}{14} : 0 = 1 : 1$, so $\mathbb{P}(BB) = 1/2$. The day of the week is now irrelevant: you have simply learned the sex of one specific child, and the other child is an independent coin flip.

Sanity check by simulation (mental or code): enumerate all 196 outcomes (we did this in Python: 27 in B, 13 of them BB). For the follow-up, the random-child protocol makes BB and mixed families equally likely to produce the report, giving 1/2.

What the interviewer is testing: whether you (a) build an explicit sample space, (b) notice that the answer depends on the protocol that generated the information, and (c) can sanity-check via limiting cases.

---

2. Derivatives: Forwards vs Futures

2.1 The contracts

A **forward** is a bilateral OTC agreement to exchange an asset at a fixed price K on a future date T. No cash changes hands at inception (for K at the fair forward), and all P&L is realised at T. Counterparty credit risk is real, mitigated by CSAs (collateral agreements) and, for some products, central clearing.

A **future** is an exchange-traded, standardised forward with **daily mark-to-market**. Each day the exchange sets a settlement price and your account is credited or debited the change (variation margin). The clearing house (e.g. CME Clearing, HKEX's HKCC) becomes the counterparty to both sides, so credit risk is replaced by liquidity risk: you must fund margin calls in cash, today.

	Forward	Future
Venue	OTC (bank to client, interbank)	Exchange
Terms	Any notional, any date	Standard size, fixed IMM-type dates
Settlement of P&L	At maturity	Daily (variation margin)
Credit risk	Bilateral (CSA helps)	Clearing house
Typical users	Corporates hedging specific cashflows (an exporter selling USD 3M forward), banks	Speculators, macro funds, CTAs, banks hedging, anyone wanting liquidity and anonymity

In FX specifically, the OTC forward market (outright forwards and FX swaps) dwarfs FX futures. CME FX futures are mostly used by funds and non-bank participants who want exchange access.

2.2 Cost of carry: pricing a forward by replication

Claim. For an asset with spot S_0, continuously compounded rate r, and continuous yield q (dividend yield, foreign interest rate, or negative storage cost),

$$F_0 = S_0\, e^{(r-q)T}.$$

Proof (cash-and-carry). If $F_0 > S_0 e^{(r-q)T}$: borrow $S_0 e^{-qT}$, buy e^{-qT} units of the asset, reinvest the yield so you hold exactly 1 unit at T, and sell forward at F_0. At T deliver the unit, receive F_0, repay $S_0 e^{(r-q)T}$. Riskless profit. The reverse inequality is arbitraged by shorting the asset and lending (reverse cash-and-carry). Hence equality. Note we never used a model for S_T: this is pure static replication.

FX version (covered interest parity). Quote S = units of domestic (quote) currency per 1 unit of foreign (base) currency. For USDJPY, base is USD and quote is JPY, so "foreign" in the formula is USD. With simple money-market rates and their day counts:

$$F = S \times \frac{1 + r_{\text{quote}}\,\tau_{\text{quote}}}{1 + r_{\text{base}}\,\tau_{\text{base}}}.$$

Forward points $= (F - S)$ scaled to pips. If the base currency has the higher rate, the forward is *below* spot (points negative): the high-yielder trades at a forward discount, which exactly offsets its carry advantage.

2.3 Worked scenario 1: a USDJPY 3M outright and an arbitrage

Market (illustrative):

- USDJPY spot $S = 148.50$
- USD 3M deposit rate 4.00% (ACT/360), JPY 3M rate 0.75% (ACT/365)

- Tenor: 91 days

Fair forward:

$$F = 148.50 \times \frac{1 + 0.0075 \times 91/365}{1 + 0.04 \times 91/360} = 148.50 \times \frac{1.0018699}{1.0101111} = 147.2884.$$

Forward points $= (147.2884 - 148.50) \times 100 \approx -121.2$ pips (quoted as about -121.2, i.e. the forward is 1.212 yen below spot). USD yields more, so USD trades at a forward discount.

A Japanese exporter expecting USD 1M of receivables in 3 months sells USD forward at 147.29 and locks in JPY 147.29M. Note they get *fewer* yen than at spot. That is not a "cost"; it is the interest differential they would have earned by converting today and depositing yen versus waiting.

Arbitrage check. Suppose a dealer is (mistakenly) showing a 3M outright of **147.60**, above fair value. USD is too expensive forward, so sell USD forward and buy it synthetically:

1. Borrow JPY 148,500,000 for 91 days at 0.75%. Owe at T: $148{,}500{,}000 (1 + 0.0075 /365) = $ JPY 148,777,674.66.
2. Sell the JPY spot for USD 1,000,000; deposit at 4.00%. Receive at T: $1{,}000{,}000 (1 + 0.04/360) = $ USD 1,010,111.11.
3. Sell USD 1,010,111.11 forward at 147.60: receive JPY 149,092,400.00.
4. Profit at T: $149{,}092{,}400.00 - 148{,}777{,}674.66 = $ **JPY 314,725.34**, riskless (ignoring credit and bid-offer).

In practice, CIP has shown persistent deviations since 2008 (the **cross-currency basis**), because bank balance sheet is costly and the arbitrage consumes it. Worth mentioning in a bank-markets interview: the "risk-free" arb is not free if it uses capital.

2.4 Mechanics of futures margining

- **Initial margin (IM):** collateral posted to open, set by the clearing house's risk model (CME uses SPAN-type methods; it is roughly a 1 to 2 day 99% loss).
- **Maintenance margin:** if your account falls below it, you get a margin call to restore the balance to the *initial* level.
- **Variation margin (VM):** daily cash flow equal to (today's settlement − yesterday's settlement) × contract size × number of contracts.

2.5 Worked scenario 2: EUR/USD futures margin calls

You buy 10 CME EUR/USD futures (contract size EUR 125,000, quoted in USD per EUR) at **1.0850**. Assume IM = USD 2,500 and maintenance = USD 2,300 per contract (illustrative). So you post USD 25,000; maintenance level USD 23,000.

Day	Settle	Daily VM ($\Delta F \times 125{,}000 \times 10$)	Balance after VM	Margin call
0	1.0850		25,000	
1	1.0820	$-3{,}750$	21,250	3,750 (back to 25,000)
2	1.0790	$-3{,}750$	21,250	3,750
3	1.0835	$+5{,}625$	30,625	0
4	1.0880	$+5{,}625$	36,250	0

Total P&L $= (1.0880 - 1.0850)000 = $ **USD 3,750**, same as a forward. You deposited $25{,}000 + 7{,}500 = 32{,}500$ and hold $36{,}250$: the difference is the 3,750 profit. But you had to find USD 7,500 of cash on days 1 and 2 even though the trade ended up a winner. That timing difference is the whole story of the next section, and of many blow-ups (Metallgesellschaft, 1993: a hedged long-dated oil position whose short-dated futures hedge generated over a billion dollars of margin calls; LTCM-style funding squeezes).

2.6 Why futures and forward prices differ: the convexity effect

With deterministic interest rates, futures price = forward price (Cox, Ingersoll and Ross, 1981). With stochastic rates they differ. Under the risk-neutral measure $\mathbb{Q}$ with discount factor $D_T = e^{-\int_0^T r_s ds}$ and zero-coupon bond $P(0,T) = \mathbb{E}^{\mathbb{Q}}[D_T]$:

- Futures price: $F^{\text{fut}} = \mathbb{E}^{\mathbb{Q}}[S_T]$ (daily settlement makes the futures price a $\mathbb{Q}$-martingale).
- Forward price: $F^{\text{fwd}} = \dfrac{\mathbb{E}^{\mathbb{Q}}[D_T S_T]}{P(0,T)}$.

Therefore

$$F^{\text{fwd}} - F^{\text{fut}} = \frac{\operatorname{Cov}^{\mathbb{Q}}(D_T, S_T)}{P(0,T)}.$$

Intuition. If the asset is positively correlated with rates, then D_T is negatively correlated with S_T and $F^{\text{fut}} > F^{\text{fwd}}$. Why: a long futures holder receives VM when S rises, which is when rates are high, so the cash is reinvested at high rates; losses arrive when rates are low and are financed cheaply. That systematic advantage gets priced in, pushing the futures price up. The effect is tiny for 3M FX but large for long-dated interest rate futures (Eurodollar/SOFR futures convexity adjustment, Curriculum item 19).

2.7 What can go wrong

- **Liquidity risk:** a perfectly hedged position can still go bust funding VM (MG 1993).
- **Basis risk:** hedging a specific date or asset with a standard contract leaves mismatch.

- **Delivery quirks:** physical delivery, delivery options (Treasury futures, coming soon), and squeezes near expiry.
- **Credit risk on forwards:** an in-the-money forward is an exposure to your counterparty (hence CVA).
- **Rolling:** long-dated hedges via short-dated futures pay the roll (contango/backwardation) each time.

2.8 Likely interview questions

1. **USD rates are higher than JPY rates. Is USDJPY forward above or below spot, and why?** Below. By CIP, $F = S(1 + r_{JPY}\tau)/(1 + r_{USD}\tau) < S$. If the forward were at or above spot you could borrow yen, buy dollars, earn the higher USD rate and lock the conversion back, a riskless profit. A forward is not a forecast.
2. **When is the futures price above the forward price?** When the underlying is positively correlated with short rates: VM gains are reinvested at high rates and losses financed at low rates. Formally $F^{\text{fwd}} - F^{\text{fut}} = \text{Cov}^{\mathbb{Q}}(D_T, S_T)/P(0,T)$. Equal if rates are deterministic.
3. **You are long futures as a hedge of a long-dated short forward commitment. What is your biggest risk?** Funding/liquidity: if prices move against the futures leg you pay VM now while the offsetting gain on the forward is only realised later. Also basis and roll risk.

---

3. Algorithms & Data Structures: Prefix Sums + Hashing

3.1 Problem

> You are given an array `pnl` of n daily P&L numbers (integers, possibly negative) and an integer K. Count the number of **contiguous** windows `pnl[i..j]` whose total is exactly K.
>
> Constraints: $1 \le n \le 2 \times 10^5$, $-10^4 \le \text{pnl}[i] \le 10^4$, $-10^9 \le K \le 10^9$.
>
> **Example 1:** `pnl = [1, 1, 1], K = 2` → `2` (windows `[0..1]`, `[1..2]`).
>
> **Example 2:** `pnl = [3, -1, 2, -2, 4, -3], K = 2` → `4` (worked through in the table below).
>
> **Example 3:** `pnl = [0, 0, 0], K = 0` → `6` (every one of the 6 windows).

(This is LeetCode 560, “Subarray Sum Equals K”, a staple at HRT, Citadel and Optiver screening rounds, often dressed up with P&L or volume.)

3.2 Brute force

Enumerate every start i, extend j to the right keeping a running sum, and count hits. That is $\binom{n+1}{2} = O(n^2)$ windows with $O(1)$ work each (if you keep the running sum; $O(n^3)$ if you re-sum naively). For $n = 2 \times 10^5$, that is 2×10^{10} operations: too slow.

3.3 The insight: turn "window sums" into "differences of prefix sums"

Define prefix sums $P_0 = 0$ and $P_j = \text{pnl}[0] + \cdots + \text{pnl}[j-1]$. Then

$$\text{sum}(\text{pnl}[i..j-1]) = P_j - P_i.$$

So we want the number of pairs $i < j$ with $P_j - P_i = K$, i.e. $P_i = P_j - K$.

Scan j from left to right. At step j, the question "how many earlier prefixes equal $P_j - K$?" is answered in $O(1)$ by a hash map from prefix value to frequency. Then insert P_j. Each element is processed once: $O(n)$ **expected time,** $O(n)$ **space.**

Why not two pointers / sliding window? That works only when all values are non-negative (window sum is monotone in the window's endpoints). With negative P&L, extending a window can decrease its sum, so the monotonicity that sliding windows rely on is gone. Prefix sums + hashing does not care about signs.

Verify Example 2. `pnl = [3, -1, 2, -2, 4, -3]`, prefixes $P = [0, 3, 2, 4, 2, 6, 3]$. For each $j \geq 1$ count earlier $P_i = P_j - 2$:

j	P_j	**need** $P_j - 2$	**earlier matches**	**window(s)**
1	3	1	0	
2	2	0	1 (P_0)	`[3,-1]`
3	4	2	1 (P_2)	`[2]`
4	2	0	1 (P_0)	`[3,-1,2,-2]`
5	6	4	1 (P_3)	`[-2,4]`
6	3	1	0	

Total **4**. (So the windows are `[3,-1]`, `[2]`, `[3,-1,2,-2]` and `[-2,4]`.)

Two subtle points.

1. **Seed the map with $P_0 = 0$ (count 1).** Otherwise you miss windows that start at index 0.
2. **Look up before inserting.** If you insert P_j first and $K = 0$, you would count the empty window $P_j - P_j$.

Overflow. Prefix sums can reach $2 \times 10^5 \times 10^4 = 2 \times 10^9 > 2^{31} - 1$, and the count of windows can be up to $n(n+1)/2 \approx 2 \times 10^{10}$. Both must be `long long`.

3.4 C++ solution

```cpp
#include <unordered_map>
#include <vector>

long long countWindows(const std::vector<int>& pnl, long long k) {
    std::unordered_map<long long, long long> seen;  // prefix sum -> how many times seen
    seen.reserve(pnl.size() * 2 + 1);
    seen[0] = 1;                                     // the empty prefix

    long long prefix = 0;
    long long count = 0;
    for (int x : pnl) {
        prefix += x;
        auto it = seen.find(prefix - k);             // earlier prefixes P[i] with P[j]-P[i]=k
        if (it != seen.end()) count += it->second;
        ++seen[prefix];                              // record current prefix AFTER the lookup
    }
    return count;
}
```

The full file `2026-10-13.cpp` adds a `main` with the examples above plus 2,000 random arrays cross-checked against the $O(n^2)$ brute force. Compile with `g++ -std=c++17 -O2 -Wall 2026-10-13.cpp`.

3.5 Walkthrough of the C++

- **`const std::vector<int>& pnl`**: pass by *const reference*. Passing `std::vector<int>` by value would copy all n elements on every call. The `&` makes it an alias to the caller's vector; `const` promises (and the compiler enforces) that we do not modify it. Rule of thumb: pass anything bigger than a couple of machine words by `const&` unless you need your own copy.
- **`std::unordered_map<long long, long long>`**: a hash table (average $O(1)$ insert and lookup, worst case $O(n)$ if many keys collide). `std::map` would be a red-black tree with $O(\log n)$ operations and sorted keys, which we do not need.
- **`seen.reserve(...)`**: pre-allocates enough buckets so the table never rehashes during the loop. Rehashing is $O(\text{size})$ and allocates memory; reserving up front is a cheap, common performance habit (and a low-latency talking point).
- **`seen[0] = 1;`**: `operator[]` returns a reference to the value for key `0`, *creating it with value-initialisation (0) if absent*. Here we create it and assign 1.
- **`for (int x : pnl)`**: a range-based for loop. It is sugar for iterating from `pnl.begin()` to `pnl.end()`, copying each element into `x`. Copying an `int` is free; for heavier elements you would write `const auto& x`.
- **`auto it = seen.find(prefix - k);`**: `find` returns an *iterator*, a lightweight handle pointing at a key-value pair, or `seen.end()` if the key is absent. `auto` lets the compiler deduce the long type `std::unordered_map<long long,long long>::iterator`. We use `find` rather than `seen[prefix - k]` deliberately: `operator[]` would *insert* a zero entry for every missing key, bloating the map.

- **`it->second`**: the element an unordered_map iterator points at is a `std::pair<const long long, long long>`. `->first` is the key (const, since changing it would corrupt the hash table), `->second` is the mapped value.
- **`++seen[prefix];`**: here we *want* `operator[]`'s insert-if-missing behaviour: if `prefix` is new it is created as 0 then incremented to 1; otherwise its count goes up by one. Pre-increment `++x` is the idiomatic choice (for iterators it can avoid a copy).
- **`long long` everywhere for sums and counts**: see the overflow analysis above. Mixing `int x` into `long long prefix` is safe because `x` is promoted before the addition.

In `main`, note `struct Test { ... };` with brace initialisation `{{1, 1, 1}, 2, 2}` (aggregate initialisation), `const auto& t : tests` to iterate without copying each test case, and `std::mt19937` with `std::uniform_int_distribution` from `<random>`: the modern replacement for `rand() % n`, which is both statistically biased and low quality.

3.6 Variations you should be ready for

- **Count windows with sum divisible by m:** key the map by $P_j \bmod m$ (normalise negatives with `((p % m) + m) % m`) and count equal residues.
- **Longest window with sum exactly K:** store the *first* index at which each prefix appears instead of a count.
- **All values non-negative:** two pointers in $O(1)$ extra space.
- **2D version (count submatrices summing to K):** fix a pair of rows, collapse columns into a 1D array, apply today's algorithm: $O(R^2 C)$.

Tomorrow: linearity of expectation and indicator tricks, Treasuries (quoting in 32nds, accrued interest), and two pointers / sliding window.